

IMAGE_GEN_PROVIDER

HelixLLM Image-Generation Provider — Design Spike (P3-T2 / GPU BURST tier)

Status	DESIGN (spike before implementation, §11.4.6 — do NOT code the provider until this is agreed AND a burst window is scheduled; see §0)
Scope	The GPU image-generation capability — a Burst-tier (single-owner §11.4.119) workload on the single RTX 5090 · 32 GB. Unlike the CPU embeddings/translation spikes, image models mostly do NOT co-reside with the resident coder fleet
Owns	Implementation-plan item P3-T2 (04_implementation_plan.md:80)
Created	2026-07-07 · Revision 1 · Track (T1/main) · Branch feature/helixllm-full-extension
Grounding	submodules/helix_llm/docs/VRAM_BROKER.md §2 (Burst tier) + internal/vrambroker/broker.go (the a12df57 broker CORE that now EXISTS: Acquire/Release/Budget, ClassImage, single-owner + fail-closed over-budget refusal) · submodules/helix_llm/docs/API_CONTRACT.md §2/§4 (gateway /v1 group) · docs/research/07.2026/00_master/04_implementation_plan.md P3-T2 · sibling EMBEDDINGS_PROVIDER.md + TRANSLATION_PROVIDER.md (structure mirror)

Anti-bluff (§11.4.6): every VRAM / latency / step-count figure in this document that is not a directly-quoted upstream value is an **estimate to be measured** ((EST – measure)). `nvidia-smi deltas + wall-clock` captured under `docs/qa/<run-id>/image_gen/` are the real budget; no benchmark below is a captured measurement. Every co-reside-vs-burst verdict **MUST** be re-confirmed against a live Budget () . free read before any PASS.

0. Why this is DESIGN-only, and why it is a BURST workload (honest §11.4.6)

The embeddings (P3-T6) and translation (P3-T5) CPU spikes ship **before** the P0/P1 GPU chain because they take a **zero-byte VRAM lease** (VRAM_BROKER.md §2 CPU-only tier). Image generation is the **opposite case** — it is the canonical **Burst tier** member (VRAM_BROKER.md §2: “*Burst (single-owner §11.4.119) ... FLUX image-gen, WAN/LTX video-gen ... started per-job, never co-resident with another burst; may require pausing warm tier*”).

The live-state constraint (the reason this is DESIGN, not impl): the RTX 5090 (32 GB) currently holds the resident coder fleet — **~19.4 GB used, ~12.7 GB free** (VRAM_BROKER.md §2 resident tier ≈ 18 GB weights + 10–12 GB KV). The flagship image models do NOT fit that free residual:

- **FLUX.1-dev / schnell at full bf16** needs **~50 GB of RAM/VRAM to load all modelling components** (HF diffusers memory doc, §1) — it does not fit the 32 GB card at all without CPU offload, let alone the free 12.7 GB.
- **FLUX fp8** (ComfyUI / optimum-quanto) drops the transformer to ~12 GB but, with the T5-XXL text encoder + CLIP + VAE + working set resident, peaks at **~16–20 GB (EST)** — still **over** the free 12.7 GB.

So the flagship path requires a **full GPU burst** — pausing the warm/coder tier to free the card — which is an **operator-gated** decision (VRAM_BROKER.md §2: “*may require pausing warm tier*”). We will **not** take that decision now; this document is the **exact, ready-to-implement spec** so that when a burst window is scheduled, implementation is a mechanical follow-through, not a fresh design.

A **smaller, heavily-quantised fallback DOES co-reside** in the free ~12.7 GB (§1.5) — that lane makes image-gen available even when no burst window is scheduled, and is the recommended first thing to actually build.

Honest boundary vs the plan (§11.4.6).

`04_implementation_plan.md`: 80 specifies P3-T2 as “*FLUX.1-dev / schnell via ComfyUI; /v1/images/generations bridge. Signature: real image, not stub (dimensions + non-trivial content check).*” This document (a) keeps ComfyUI + FLUX as the reuse choice (§11.4.74), (b)

strengthens the “non-trivial content check” signature into a CLIP-text-image-similarity semantic oracle with a golden-bad self-validation (§5, per §11.4.107(10) — a “dimensions + not-all-black” check is a §11.4 bluff: a random-noise or wrong-prompt image passes it), and (c) adds the explicit Burst-tier broker integration the plan line does not spell out. Flagged, not a contradiction.

1. Engine choice — evidence-based comparison

1.1 The candidates

#	Engine	Serves image-gen how?	Fits free ~12.7 GB (co-reside)?	Model coverage	New engine to the tree?
A	ComfyUI (graph server: POST / prompt + /ws + /history/{id} + /view)	Programmatic workflow-graph API; the gateway bridges OpenAI-images → a ComfyUI graph	No for FLUX fp8 (~16–20 GB peak) — needs a burst ; SDXL co-resides	FLUX.1-dev/schnell, SDXL/SD3.5, LoRA, ControlNet — broadest	Container only (no Go dep); the plan already names it
B	stable-diffusion.cpp (leejet/stable-diffusion.cpp) + a thin FastAPI shim	GGUF/quantised inference; needs a LibreTranslate-style OpenAI shim (like the NLLB lane)	Yes for FLUX-schnell q4 + SDXL (T5 on CPU) — the co-reside lane	FLUX.1-dev/schnell, SDXL/SDXL-Turbo, SD3/SD3.5; GGUF q4/q8; CPU(AVX/AVX2/AVX512)+CUDA	New shim container (same family as the NLLB shim)
C	diffusers (Python FluxPipeline + a FastAPI shim)	Python lib; needs a custom OpenAI-images shim	fp8 (optimum-quanto) <16 GB but peak still >12.7; needs a burst	FLUX, SDXL, SD3, every HF pipeline; every offload/quant knob	New Python shim + heavier deps
D				remote	

#	Engine	Serves image-gen how?	Fits free ~12.7 GB (co-reside)?	Model coverage	New engine to the tree?
	Hosted API (OpenAI gpt-image, others)	Native — but a bespoke remote adapter	N/A (remote)		New remote dep + secret + egress — rejected as default

1.2 Decision

Primary engine: ComfyUI serving FLUX.1-schnell (flagship), on a full GPU burst (operator-gated). ComfyUI is the reuse choice the plan already names (§11.4.74), exposes a real programmatic API (POST /prompt → /ws progress → /history/{id} → /view), and covers FLUX.1-schnell (Apache-2.0), FLUX.1-dev (higher quality, non-commercial), SDXL, LoRA, and ControlNet behind ONE backend and the SAME /v1/images/generations gateway route. **FLUX.1-schnell is the default flagship model** (Apache-2.0 + 1–4 steps ⇒ the shortest possible burst); **FLUX.1-dev is the higher-quality lane** behind the same engine + route (heavier, non-commercial licence — §1.4).

Documented smaller-fits-more-often fallback: stable-diffusion.cpp (GGUF) via a thin OpenAI-images shim, serving FLUX.1-schnell-q4 (T5 encoder on CPU) OR SDXL — which CO-RESIDES in the free ~12.7 GB, no coder pause. This is the lane that makes image-gen available **without** a scheduled burst window, and — being a GGUF/CPU-capable engine in the same family as the NLLB CTranslate2 shim — reuses a pattern already in the tree. **Recommendation: build the co-reside sd.cpp fallback FIRST** (available today, no operator gate), and land the ComfyUI+FLUX flagship burst path when a burst window is scheduled.

1.3 Justification (cited, LATEST upstream docs — §11.4.99 / §11.4.150)

1. **FLUX.1-schnell is the right flagship for a BURST: permissive licence + fewest steps.** It is “a 12 billion parameter rectified flow transformer”, “Released under the apache-2.0 licence, the model can be used for personal, scientific, and commercial purposes”, and “can generate high-quality images in only 1 to 4 steps” (trained with “latent adversarial diffusion distillation”, guidance_scale=0, max_sequence_length=256). Fewest steps = the shortest GPU-burst occupancy = the least time the coder tier is paused.

- Sources: [black-forest-labs/FLUX.1-schnell \(HF model card\)](#) (accessed 2026-07-07); [diffusers — FLUX pipeline / memory](#) (accessed 2026-07-07).
- 2. FLUX.1-dev is the higher-quality lane, but non-commercial + heavier.**
Also “a 12 billion parameter rectified flow transformer”, governed by the “FLUX.1 [dev] Non-Commercial License”, and the reference implementation runs “num_inference_steps=50” with guidance_scale=3.5 — ~12× the schnell step count and a licence that blocks commercial ship. It is the same engine + same route (a config model-swap), so the CPU → GPU-quality migration is a model-id change, not an ecosystem change. Sources: [black-forest-labs/FLUX.1-dev \(HF model card\)](#) (accessed 2026-07-07); [diffusers — FLUX pipeline](#) (accessed 2026-07-07).
 - 3. The full-FLUX memory wall is why this is a burst, quantified from upstream.** diffusers states “Flux is a very large model and requires ~50GB of RAM/VRAM to load all the modeling components.” The documented sub-16 GB paths — optimum-quanto fp8 (“< 16GB VRAM”), bitsandbytes 8-bit, and enable_sequential_cpu_offload() (“For Low VRAM: 4-32GB”) + vae.enable_slicing()
 - vae.enable_tiling() — all trade throughput for footprint and still peak **above the free 12.7 GB** with the T5 encoder resident (§1.5). This is the evidence base for “flagship FLUX ⇒ full burst, coder paused.”
Source: [diffusers — FLUX memory optimisation](#) (accessed 2026-07-07).
 - 4. stable-diffusion.cpp is the genuine co-reside fallback: GGUF + CPU text encoding.** Its feature list explicitly supports “FLUX.1-dev/FLUX.1-schnell”, “SDXL, SDXL-Turbo”, “SD3/SD3.5”, the “GGUF (.gguf)” weight format, and both “CPU (AVX, AVX2 and AVX512 support for x86 architectures)” and “CUDA” backends. Running the FLUX transformer at GGUF **q4** on the GPU while keeping the T5-XXL text encoder on **CPU** drops the resident GPU footprint to ~7–9 GB (EST) — which fits the free 12.7 GB with the broker’s 2 GB headroom (§1.5). Being GGUF + CPU-capable, it is the direct image analogue of the NLLB CTranslate2-int8 CPU shim (TRANSLATION_PROVIDER.md §1) — a family already in the tree. Source: [leejet/stable-diffusion.cpp \(README\)](#) (accessed 2026-07-07).
 - 5. ComfyUI is a real programmatic backend, not just a UI.** It exposes POST /prompt (“submit a prompt to the queue” → prompt_id), a /ws WebSocket (execution_start / executing / progress / executed), GET /history/{prompt_id} (“retrieve the queue history ... containing execution results”), and GET /view (“view an image”) — the four routes the gateway bridges an OpenAI-images request onto (§3). ComfyUI’s own tutorial documents the FLUX fp8 lane (“fp8 checkpoint version reduces VRAM needs”; t5xxl_fp8_e4m3fn.safetensors “when your VRAM is low” vs t5xxl_fp16 “recommended for >32GB VRAM”; required files clip_l.safetensors, t5xxl_*.safetensors, ae.safetensors). Sources: [ComfyUI — server API routes](#) (accessed 2026-07-07); [ComfyUI — FLUX.1 text-to-image tutorial](#) (accessed 2026-07-07).

6. **SDXL is the safe-licence, comfortably-co-resident fallback model.** SDXL base is a base+refiner “*ensemble of experts pipeline*” under the “*CreativeML Open RAIL++-M License*” (permissive) with `pipe.enable_model_cpu_offload()` for VRAM-limited hosts. Its ~2.6 B-param UNet at fp16 (~5–6 GB) + text encoders
 - VAE + working set peaks at ~7–9 GB (EST) — the most reliable co-reside candidate, and the fallback-of-the-fallback when a FLUX pair is unavailable. Source: [stabilityai/stable-diffusion-xl-base-1.0 \(HF model card\)](#) (accessed 2026-07-07).
7. **Why the hosted-API lane (D) is rejected as default.** A hosted image API requires a committed key (§CONST-042 / §11.4.10 secret-leak surface) + third-party egress of user prompts, and defeats the offline/self-host posture. It MAY be added later as an opt-in, config-injected, key-from-`.env` provider descriptor — never the default (mirrors `TRANSLATION_PROVIDER.md` §1.3(6)).

1.4 Model selection (config value, never hardcoded — §CONST-046 / §11.4.35)

Model	Params	Steps	Licence	Fits free ~12.7 GB?	Role
black-forest-labs/FLUX.1-schnell (fp8, ComfyUI)	12B	1–4	Apache-2.0	No (~16–20 GB peak (EST)) → full burst	Default flagship — permissive + fewest-step burst
black-forest-labs/FLUX.1-dev (fp8, ComfyUI)	12B	~50	Non-commercial	No (~16–20 GB peak (EST)) → full burst	Higher-quality lane (heavier, licence-gated Q5)
FLUX.1-schnell-Q4 (GGUF, stable-diffusion.cpp, T5 on CPU)	12B → q4	1–4	Apache-2.0	Yes (~7–9 GB (EST)) → co-reside	Co-reside fallback — image-gen with no coder pause
stabilityai/stable-diffusion-	~2.6B UNet	25–40	CreativeML OpenRAIL++-M	Yes (~7–9 GB (EST)) → co-reside	Fallback model — safe

Model	Params	Steps	Licence	Fits free ~12.7 GB?	Role
x1-base-1.0 (fp16)					licence, reliably co-resident

Licence note (§11.4.6, flag for review — Q5): FLUX.1-dev is **non-commercial**. If HelixLLM ships image-gen commercially, the dev lane is blocked and the **Apache-2.0 FLUX.1-schnell + OpenRAIL++ SDXL** lanes are load-bearing. Documentation flag, not a resolution.

Recommendation: default flagship FLUX.1-schnell fp8 via ComfyUI (burst); default co-reside fallback SDXL (safe licence, reliably fits) OR FLUX.1-schnell-Q4 GGUF via sd.cpp (higher quality, tighter fit) — chosen on measured Budget () . free at admission time (§2), never a static guess.

1.5 VRAM budget — ESTIMATES to be measured (§11.4.6), fit-vs-burst verdict

All figures (EST — measure); replace with on-card nvidia-smi deltas + p50/p95/p99 wall-clock captured under docs/qa/<run-id>/image_gen/ before any PASS. Free-VRAM baseline: coder resident $\Rightarrow$ ~12.7 GB free (§0). Broker headroom = 2 GiB (broker.go:13 HeadroomBytes), so a co-reside model must fit need + 2 GiB $\leq$ 12.7 GiB $\Rightarrow$ need $\leq$ 10.7 GiB.

Config	Transformer/UNet (EST)	Text enc (T5 / CLIP) (EST)	VAE + working (EST)	Peak resident (EST)	Verdict vs free 12.7 GB
FLUX.1-dev/ schnell bf16 full	~24 GB	~9.5 GB (T5 bf16)	~2 GB	~50 GB (diffusers: “~50GB ... to load all ... components”)	Full burst + CPU offload — exceeds 32 GB card
FLUX.1-schnell/ dev fp8 (ComfyUI)	~12 GB	~4.7 GB (T5 fp8, resident)	~2 GB	~16–20 GB	Full burst (coder paused) — over

Config	Transformer/ UNet (EST)	Text enc (T5 / CLIP) (EST)	VAE + working (EST)	Peak resident (EST)	Verdict vs free 12.7 GB
					12.7 GB free
FLUX.1- schnell q4 GGUF (sd.cpp, T5 on CPU)	~7 GB	~0.25 GB (CLIP only on GPU)	~1.7 GB	~7–9 GB	Co- reside ✓ (need+2 ≤ 12.7)
SDXL fp16 (sd.cpp / ComfyUI)	~5.5 GB	~1.4 GB	~1.7 GB	~7–9 GB	Co- reside ✓

Latency (EST): FLUX-schnell 4-step burst ~2–8 s per 1024² image on the 5090; FLUX-dev 50-step ~20–60 s; SDXL 30-step ~3–10 s; sd.cpp q4 with CPU T5 slower on the text-encode leg. **These numbers gate nothing until measured.**

2. VRAM-broker integration — the burst lease (§11.4.119)

The broker CORE **already exists** — `internal/vrambroker/broker.go` (commit a12df57, verified 2026-07-07). Image-gen is class **ClassImage** (`broker.go:21` `ClassImage Class = "image" // burst tier – single-owner` (§11.4.119)), and `Class.IsBurst()` is true for it (`broker.go:29`). The provider acquires a burst lease around every generation job:

```
// pseudo-flow in the image-gen provider / gateway handler
lease, err := broker.Acquire(ctx, vrambroker.ClassImage,
    needBytes) // needBytes = measured model footprint
if err != nil {
    // ErrBurstInUse → another image/video burst is live → 429/
    queue (§2.2)
    // ErrBudgetExceeded → over budget while coder resident → the
    operator-gated
    // "pause warm tier" decision (§2.3), NOT
    an OOM (fail-closed)
    // ErrThermalUnsafe / ErrBudgetUnavailable → 503, honest
    (§11.4.133)
    return mapBrokerErrorToHTTP(err)
```



```

}
defer lease.Release() // idempotent (broker.go:47) – frees the
    single-owner slot
img := backend.Generate(ctx, req) // ComfyUI /prompt ... or
    sd.cpp shim

```

2.1 What the existing broker already enforces (reuse, don't reimplement §11.4.74)

- **Single-owner burst (§11.4.119).** Acquire refuses a second burst lease with `ErrBurstInUse` **before** the budget read — “*Single-owner enforcement for burst classes ... no second burst lease may be live while one is. Checked BEFORE the budget read so a contended burst is rejected deterministically.*” (broker.go:147-149). Image and video **cannot** run concurrently — exactly the §11.4.119 partitioning the plan requires.
- **Fail-closed over-budget refusal (NEVER an OOM).** Admission is `admit(free, needBytes, headroom) = free >= needBytes + headroom` on the **measured** `nvidia-smi free` (broker.go:161-183, budget.go:readNvidiaSMI); an unreadable budget returns `ErrBudgetUnavailable` (fails closed, broker.go:157-160); an over-budget request returns `ErrBudgetExceeded` (broker.go:161-163). This is the anti-bluff core the §1.1 mutation disables (mutation_test.go).
- **Thermal/power safety gate (§11.4.133).** `WithThermalGuard` injects a temperature/power probe consulted before admission (broker.go:82, 151-154); refusing returns `ErrThermalUnsafe`. Image-gen is a heavy, sustained-power workload — this gate is load-bearing for a burst.
- **Idempotent release.** `Lease.Release()` frees the single-owner slot exactly once (broker.go:47-55) — a `defer lease.Release()` around the job hands the card back deterministically.

2.2 Co-reside (small model) — no coder pause needed

For the **co-reside fallback** (SDXL / FLUX-schnell-q4, `needBytes ≈ 8 GiB`), `Acquire(ctx, ClassImage, 8 GiB)` **succeeds while the coder stays resident**: $8 + 2 \leq 12.7 \Rightarrow \text{admit returns true}$. It is still a **single-owner** burst (no concurrent image/video), but it does **not** require pausing the coder — the job runs in the free residual and `Release` hands it back. This is the lane that ships first (no operator gate).

2.3 Pause-warm-tier (large model) — operator-gated, DESIGN-only

For the **flagship** (FLUX fp8, `needBytes ≈ 18 GiB`), `Acquire(ctx, ClassImage, 18 GiB)` **fails closed with `ErrBudgetExceeded`** while the coder is resident ($18 + 2 > 12.7$). The design intent (VRAM_BROKER.md §2/§4: “*may*

require pausing warm tier ... pause the warm tier, run, and the warm tier resumes”)
is to evict/park the warm+coder tier to free the card, run the burst, then resume.

Honest boundary (§11.4.6): the broker does NOT yet implement eviction. The shipped Acquire **refuses** over-budget rather than pausing the coder (broker.go has no sleep(coder) / vLLM-Sleep-Mode / keep_alive:0 call — confirmed by reading the file). Pausing the resident coder fleet to free ~18 GB is a **high-blast-radius, operator-gated decision** (it stalls every live agent), so it is **deferred to the P1-T4 residency-scheduler implementation + an explicit operator go**, per §0. Until then, the flagship-FLUX path is **admissible only during a scheduled burst window** where the coder tier is already parked; the co-reside fallback (§2.2) covers the everyday case. This is the §11.4.101 block-only-when rule: pausing the coder is irreversible-for-the-session + high-blast-radius, so it blocks on the operator rather than being decided autonomously.

2.4 Async job model (composes the broker queue)

Because a flagship burst may **wait** (for a scheduled window, or behind a live burst), the gateway route (§3) supports an **async job** shape (submit → poll) in addition to the sync path — mirroring P3-T3 video (04_implementation_plan.md:81 “*async job endpoint*”). A queued job surfaces honestly (202 Accepted + a job id, or 429 with Retry-After on ErrBurstInUse), never a silent hang (VRAM_BROKER.md §5 starvation guard).

3. API contract — OpenAI-images /v1/images/generations (consistent with API_CONTRACT.md)

HelixLLM registers **no** image route today — grep -rn "images/generations" submodules/helix_llm/internal returns **empty** (verified 2026-07-07). This provider **adds** POST /v1/images/generations under the gateway /v1 group so it inherits the API-key middleware + rate-limit + security-headers uniformly (API_CONTRACT.md §2/§3, router.go:63-68) — the same reviewed placement the translation spike chose for /v1/translate (TRANSLATION_PROVIDER.md §2).

3.1 Request — OpenAI images shape

```
POST /v1/images/generations
Authorization: Bearer <key>           # gateway /v1 API-key middleware (r
Content-Type: application/json
{
  "model": "helix-image",              # Helix alias → backing FLUX/SDXL m
```

```

"prompt": "a red fox reading a book under an oak tree, watercolor",
"n": 1,                                     # OPTIONAL — number of images
"size": "1024x1024",                       # OPTIONAL — e.g. 1024x1024, 1536x1024
"quality": "auto",                         # OPTIONAL — low|medium|high|auto (m
"response_format": "b64_json"             # OPTIONAL — "b64_json" (DEFAULT her
}

```

- **model** — a **Helix alias**, not a raw HF id (CONST-036/037: models come from the provider layer / LLMsVerifier, never hardcoded). Alias → backing-model map (helix-image → FLUX.1-schnell / FLUX.1-dev / SDXL) lives in HelixLLM config (env/YAML), never a source literal (§CONST-046) — adding a model = a config edit, no code change.
- **quality** — maps to a step-count/model policy in the shim (low → schnell 1-step, high → dev 50-step), never exposed as a raw step int to the client.
- **response_format** — **defaults to b64_json** (self-contained; no object store required in-tree). **url** requires a configured static-asset host (Open Q3).

Source for the request/response field names: [OpenAI — image-generation guide \(developers.openai.com\)](https://openai.com/docs/api-reference/images/create) (accessed 2026-07-07). (Negative finding §11.4.99(B): the canonical platform.openai.com/docs/api-reference/images/create returned **HTTP 403** to automated fetch on 2026-07-07 — the field list is instead grounded in the developers-guide mirror above + the in-tree API_CONTRACT.md gateway placement, which is the authoritative shape for THIS system.)

3.2 Response — OpenAI images shape

```

{
  "created": 1751840000,
  "data": [
    { "b64_json": "iVBORw0KGgoAAAANSUhEUgAA...(PNG bytes,
      base64)..." }
  ]
}

```

- **data[]** — one entry per generated image; {b64_json} (default) or {url}.
- **GET /v1/models** MUST advertise helix-image with owned_by: "helix" and an image capability flag so LLMsVerifier + clients discover it (CONST-036/040) — the capability flag is verifier-sourced, never hardcoded (P2-T3 seam).

3.3 Backend bridge — ComfyUI (primary) / sd.cpp shim (fallback)

- **ComfyUI backend**: the handler builds a FLUX/SDXL **workflow graph** from {prompt, size, quality} → POST /prompt (→ prompt_id) → waits on the /ws executed event (or polls GET /history/{prompt_id}) → fetches

the PNG via GET /view → returns it b64_json. (ComfyUI routes verified — §1.3(5).)

- **sd.cpp fallback backend:** a thin FastAPI/Go shim (the image analogue of the NLLB CTranslate2 shim, TRANSLATION_PROVIDER.md §3) wraps the sd binary / library, taking --diffusion-model <gguf> --clip_l ... --t5xxl ... --vae ... -p <prompt> --steps N and returning the PNG.

3.4 Error shape

OpenAI-error envelope, consistent with the gateway (API_CONTRACT.md §3, auth.go:58-64): {"error":

{"message": "...", "type": "invalid_request_error"}} for auth/validation; **429** + **Retry-After** when broker.Acquire returns ErrBurstInUse (another burst live); **503** ({"...", "type": "server_error"}) when the backend container is not warm OR ErrBudgetExceeded/ErrThermalUnsafe (honest “card busy / warming / thermally throttled”) — **never** a returned stub image (the bluff §5 kills).

4. Containerization — rootless podman via the containers submodule, WITH GPU

Per §11.4.76 (containers-submodule mandate) + §11.4.161 (rootless runtime) the service boots **through** vasic-digital/containers (pkg/boot / pkg/compose / pkg/health), never a hand-run podman/docker command, and never rootful. **Unlike the CPU spikes, this container claims the GPU.**

4.1 Image + run shape (illustrative — config-injected, no hardcoded host §CONST-045)

- **Primary (ComfyUI) image:** a CUDA-12.8 / sm_120 base (the P0-T2 pinned nvidia/cuda:12.8.x base, 04_implementation_plan.md:46) + ComfyUI + FLUX/SDXL weights. Pinned by digest in production (§11.4.76 clause 2).
- **Fallback (sd.cpp) image:** the P0-T2 CUDA base + the stable-diffusion.cpp build (sm_120) + the GGUF weights + the OpenAI-images shim. Also runs CPU-only (AVX/AVX2/AVX512) as a degraded lane.
- **GPU passthrough (the structural difference from the CPU spikes):** the run spec **DOES** include --device nvidia.com/gpu=all --security-opt=label=disable — the exact P0 GPU-passthrough proof (04_implementation_plan.md:44). This is the structural reason it depends on **P0** (host GPU foundation) and **P1** (the broker), whereas the embeddings/translation CPU spikes do not.
- **Model source:** weights are a §11.4.77 re-obtain artefact (gitignored; fetch_weights.sh-class script downloads from HF — matches 04_implementation_plan.md P7-T1 fetch_weights.sh). \$MODELS_DIR is

injected (env), never a literal; mounted read-only (-v \$MODELS_DIR:/models:ro).

- **Port:** a config-injected host port (ComfyUI default :8188, distinct from the coder fleet's :18434), reached by the HelixLLM gateway; --network per the containers-submodule compose spec.
- **Boot is part of the test entry point** (§11.4.76 on-demand-infra invariant): the HelixQA image bank boots the container via the submodule, waits on pkg/health (ComfyUI /system_stats or a shim /health), acquires the broker burst lease, then drives the gateway route — a short-circuit fake that skips the boot is a §11.4 violation.
- **Broker interaction:** boot/generation is gated by broker.Acquire(ctx, ClassImage, needBytes) (§2) — the container is only started/loaded when a burst lease is granted; on Release the container may be stopped (flagship, to free the card) or kept warm (co-reside).
- **Catalogue-Check (§11.4.74):** extend vasic-digital/containers@<sha> — add an image-gen (ComfyUI + sd.cpp) compose profile to the containers submodule if one does not exist; never an in-project ad-hoc compose file.

4.2 Cross-platform + resource/host-safety hygiene

- GPU passthrough is Linux/NVIDIA-specific; on a host without the 5090 the bank SKIPS-with-reason (hardware_not_present, §11.4.69) — never a faked PASS.
- Container VRAM/compute bounded by the broker lease (§2) + pkg/health; §12.6 host-memory + §11.4.133 thermal safety apply — a burst must not starve the developer host nor thermally endanger the card (the broker's ThermalGuard).

5. Anti-bluff acceptance — the ONE machine-checkable runtime signature (§11.4.108)

Definition of done for this provider: on a **clean deploy** (§11.4.108/§11.4.139 — container freshly booted via the containers submodule, broker burst lease granted, gateway pointed at it), the following single machine-checkable signature verifies and is captured to docs/qa/<run-id>/image_gen/:

RUNTIME SIGNATURE (image-gen semantic text → image correspondence). POST a **prompt pair** to POST /v1/images/generations — a **matched** prompt P = “a red fox reading a book under an oak tree, watercolor” and an **unrelated control** prompt U = “a bowl of ramen on a wooden table”. Decode each returned b64_json PNG. Compute the **CLIP (or SigLIP) text-image similarity** CLIPScore(image, caption) and assert **ALL** of: 1. **Real image, not a**

stub: the PNG decodes, has the requested dimensions, and is **non-trivial** — luminance variance / edge-density / distinct-colour count above a fixture floor (kills the all-black / solid-colour / returned- stub image; the plan’s “dimensions + non-trivial content” made mechanical).

2. **Prompt-image correspondence:** $\text{CLIPScore}(\text{img_P}, P) \geq \text{floor}$ — the generated image genuinely depicts its prompt (floor calibrated on the project’s own fixtures per §11.4.107(13)). 3. **Semantic-order margin (the anti-noise-image guard):** $\text{CLIPScore}(\text{img_P}, P) - \text{CLIPScore}(\text{img_P}, U) \geq \text{margin}$ — the image matches its OWN prompt more than an unrelated caption (a random-noise or wrong-prompt image fails this even if it passes the trivial-content check). The captured artefact is the raw /v1/images/generations JSON + the decoded PNGs + the CLIPScore matrix + PASS/FAIL verdict with its evidence path (feature class `gpu_render/image_gen`, §11.4.69).

CLIPScore is “a metric based on a modified cosine similarity between representations for the input image and the caption”, computed as $w \times \max(\cos(c, v), 0)$ with $w = 2.5$ (range $[0, 2.5]$), **reference-free**, and “achieving high correlations with human judgments” — the right oracle for “does this image depict this text” with no golden image required. Source: [CLIPScore — reference-free image-text similarity \(torchmetrics / Hessel et al. survey\)](#) (accessed 2026-07-07). This is a genuine end-to-end proof: it can only PASS if a real diffusion model produced a prompt-faithful image through the real gateway route + a live broker burst lease — impossible to satisfy with a returned stub, a solid-colour image, or a wrong-prompt/noise image.

5.1 Golden-good / golden-bad self-validation (§11.4.107(10))

The CLIPScore **analyzer itself is mutation-proofed** with a fixture set, wired into the meta-test:

- **golden-good fixture** — a captured real prompt-faithful image + its prompt where the margin genuinely holds → the analyzer **MUST** return **PASS**.
- **golden-bad fixtures** (each **MUST** return **FAIL**, proving the analyzer cannot be fooled):
 1. **solid-colour / all-black** PNG (the stub shape) → fails the non-trivial-content check (criterion 1).
 2. **random-noise** PNG → passes trivial-content but fails the prompt-correspondence floor + the semantic-order margin (criteria 2+3).
 3. **wrong-prompt** image (a real photo of ramen returned for the fox prompt) → fails the semantic-order margin $\text{CLIPScore}(\text{img}, P) < \text{CLIPScore}(\text{img}, U)$ (catches a mis-wired backend / a cached-from-a-different-request image).
 4. **wrong-dimensions** PNG → fails criterion 1’s size check.

Paired §1.1 mutation: strip the semantic-order-margin (or the non-trivial-content) assertion from the analyzer → the random-noise/solid-colour golden-bad fixture PASSES → the gate FAILs. That mutation is the mechanical proof the acceptance test is not itself a bluff.

5.2 Higher-order + resilience proofs (compose, do not replace §5)

- **Metamorphic seed determinism (§11.4.50):** same prompt + same seed → byte-identical (or CLIPScore-identical within tolerance) image across two calls; different seeds → different images with both matching the prompt.
- **Re-runability (§11.4.98):** the whole bank PASSES at -count=3 with self-cleaning state (broker lease released between runs).
- **Stress + chaos (§11.4.85):** $N \geq 10$ sequential jobs (the burst queue serialises them — throughput + p50/p95/p99 captured); a **second concurrent** image/video request MUST hit ErrBurstInUse → 429 (the §11.4.119 single-owner proof); chaos = container SIGKILL mid-generation → broker lease auto-releases (or the P1-T4 lease-TTL reaps it), gateway returns an honest 503, the card is not leaked; boundary prompts (empty, max-length, unicode, adversarial) each categorised.
- **Broker-refusal negative case (composes VRAM_BROKER.md §6):** a scripted over-budget flagship Acquire returns ErrBudgetExceeded and the gateway returns 503 — it does **NOT** OOM the card (fail-closed); paired §1.1 mutation (disable admit) → the over-commit would be granted → the broker gate FAILs.
- **Feature-class evidence (§11.4.69):** the closed sink-side taxonomy has a `gpu_render` class; this provider maps to it (an `image_gen` sub-class is a candidate addition — taxonomy is *open to additions, never contraction*). Evidence shape = the captured CLIPScore + non-trivial-content artefact above.

5.3 Four-layer verification (§11.4.108)

1. **SOURCE** — `/v1/images/generations` route + backend bridge + `ClassImage Acquire/Release` wiring + provider descriptor committed; pre-build grep gate.
2. **ARTIFACT** — the ComfyUI/sd.cpp image pulled + pinned by digest; the FLUX/SDXL weights present (`fetch_weights.sh`); `pkg/health` green; the `sm_120` GPU build present (P0-T3).
3. **RUNTIME-ON-CLEAN-TARGET** — the §5 signature verifies against a freshly-booted container **with a live broker burst lease** (not a stale one) — the definition of done. **Ready-to-run the moment a burst window is scheduled** (the operator go of §0/§2.3), or immediately for the co-reside fallback (§2.2).

4. **USER-VISIBLE** — an OpenAI-images client (HelixCode / any CLI agent / a DALL·E- compatible client) POSTs /v1/images/generations, gets a real prompt-faithful image back, and a downstream consumer renders it.
-

6. Open questions (resolve before coding)

- **Q1 Build order** — ship the **co-reside sd.cpp fallback FIRST** (available today, no operator gate, §2.2) and defer the ComfyUI+FLUX flagship burst to a scheduled window? (Leaning: yes — a real, usable image-gen lane now beats a blocked flagship.)
 - **Q2 Sync vs async route shape** — sync /v1/images/generations for the fast co-reside path + an async job endpoint (submit → poll) for the flagship burst (§2.4)? Or async-only for uniformity with P3-T3 video?
 - **Q3 response_format: "url"** — requires a configured static-asset host to serve the PNG; b64_json (default) needs none. Add the URL lane only when an asset host is configured (§CONST-045 injected, never hardcoded).
 - **Q4 Does the containers submodule already have an image-gen (ComfyUI / sd.cpp) compose profile, or is this a §11.4.74 extend PR?** (Investigate before scaffolding.)
 - **Q5 Licence** — FLUX.1-dev is non-commercial. Reconcile against HelixLLM's distribution model, or make the Apache-2.0 FLUX.1-schnell + OpenRAIL++ SDXL lanes the commercial default. Blocking for any commercial ship.
 - **Q6 Broker eviction (§2.3)** — the shipped broker **refuses** over-budget rather than pausing the coder. When P1-T4 implements the residency scheduler (vLLM Sleep Mode / keep_alive:0), the flagship-burst “pause warm tier → run → resume” path becomes autonomous within a scheduled window; until then it is operator-gated.
 - **Q7 CLIP vs SigLIP for the §5 analyzer** — CLIP ViT-L/14 (CLIPScore canonical) vs SigLIP (stronger zero-shot). Decide on measured golden-good/bad separation on the project's own fixtures (§11.4.107(13)), not the estimates here.
-

7. Composition footer — constitutional anchors touched

- **§11.4.6 (no-guessing)** — every VRAM/latency/step figure flagged (EST — measure); the licence, the broker-does-not-yet-evict boundary, and the co-reside-vs-burst verdicts all flagged, not asserted.
- **§11.4.74 (extend-don't-reimplement)** — reuse ComfyUI + FLUX (the engine/ model the plan names) + stable-diffusion.cpp (GGUF, the co-reside lane) + the

- EXISTING internal/vrambroker core + extend the containers submodule; no bespoke image server, no re-implemented admission gate.
- **§11.4.76 / §11.4.161** (containers submodule / rootless) — boot via pkg/boot+pkg/compose+pkg/health, rootless podman, WITH the GPU device.
 - **§11.4.77** (re-obtain mechanism) — model weights gitignored + fetch_weights.sh.
 - **§11.4.81** (cross-platform parity) — GPU passthrough Linux/NVIDIA-specific; no-GPU host $\Rightarrow$ honest hardware_not_present SKIP; sd.cpp CPU lane as the degraded cross-platform path.
 - **§11.4.99 / §11.4.150** (latest-source + deep multi-angle research) — engine decision cited to LATEST upstream docs across ≥ 4 distinct angles (FLUX schnell/dev model cards, diffusers memory doc, ComfyUI, stable-diffusion.cpp, SDXL, CLIPScore).
 - **§11.4.107(10)/(13)** (self-validated analyzer + fixture-calibrated thresholds) — golden-good/golden-bad CLIPScore analyzer with a solid-colour + random-noise + wrong-prompt golden-bad set; project-calibrated floor/margin.
 - **§11.4.108 / §11.4.139** (four-layer runtime-signature on a clean target) — the §5 acceptance signature is the definition of done, ready-to-run on a scheduled burst window.
 - **§11.4.119** (single-resource-owner burst) — image-gen is a ClassImage single-owner burst; concurrent image/video refused with ErrBurstInUse.
 - **§11.4.133** (thermal/power safety gates admission) — the broker's ThermalGuard is load-bearing for a sustained-power image burst.
 - **§11.4.85 / §11.4.98 / §11.4.50** (stress+chaos / full-automation / determinism).
 - **§11.4.135** (standing regression guard) — the §5 signature registers as a permanent guard.
 - **§11.4.69** (sink-side evidence taxonomy) — maps to gpu_render (candidate image_gen sub-class).
 - **CONST-036/037/040** (LLMsVerifier single source of truth; capability flags verifier-sourced) — helix-image alias + image capability from the verifier, never hardcoded.
 - **CONST-042 / §11.4.10** (no-secret-leak) — the hosted-API lane rejected as default precisely because it requires a committed key + third-party egress.
 - **CONST-046** (no hardcoded content) — model ids / host / port / alias map config-injected.

Sources verified

Deep-research 2026-07-07: - <https://huggingface.co/black-forest-labs/FLUX.1-schnell> - <https://huggingface.co/black-forest-labs/FLUX.1-dev> - <https://huggingface.co/docs/diffusers/en/api/pipelines/flux> - <https://docs.comfy.org/tutorials/flux/flux-1-text-to-image> - https://docs.comfy.org/development/comfyui-server/comms_routes - <https://github.com/leejet/stable-diffusion.cpp> - <https://huggingface.co/stabilityai/stable-diffusion-xl-base-1.0> - <https://>

developers.openai.com/api/docs/guides/image-generation - <https://arxiv.org/abs/2104.08718> (CLIPScore — reference-free image-text similarity)

(Negative finding, §11.4.99(B): platform.openai.com/docs/api-reference/images/create and lightning.ai/docs/torchmetrics/.../clip_score.html both returned HTTP 403 to automated fetch on 2026-07-07; the /v1/images/generations field list is grounded in the developers.openai.com guide mirror + the in-tree API_CONTRACT.md gateway placement, and the CLIPScore definition in the arXiv source + the torchmetrics description surfaced via web search — the authoritative shapes for THIS system.)